

AWAP-AI

AI-Driven Automated Web Application Penetration Testing System

CORE SCANNING ENGINE IMPLEMENTATION GUIDE

For AI Code Generation Systems · Version 1.0 · 2025

12 Critical Subsystems · Full Python Implementation Patterns · Production-Grade Code

■ FOR AUTHORIZED SECURITY RESEARCH USE ONLY ■

■■■ PREFACE ■■■

This document is a ground-level implementation guide for the critical subsystems of the AWAP-AI scanning engine. It is written specifically to prevent the most common failure modes that occur when AI code generators produce scanning tools: incorrect async handling, missing scope enforcement, fragile response parsing, and flawed crawl logic. Every section includes real working Python patterns, known failure modes, and production-grade implementation guidance.

Architecture, UI, and folder structure are covered in separate documents. This guide covers only the engine internals.

■■■ 1. WEB CRAWLER ENGINE — DETAILED IMPLEMENTATION ■■■

◆ 1.1 Core Crawling Principles

The crawler is the foundation of the entire scan. If it misses endpoints, nothing downstream can find them. The crawler must behave like a real browser operated by a security researcher: it follows links, submits forms, executes JavaScript, respects session state, and never visits the same URL twice.

■ Every URL must pass scope enforcement BEFORE the request is made – not after. Visiting out-of-scope URLs even once is a compliance failure.

◆ 1.2 Handling Modern JavaScript Websites

Static HTML crawling is insufficient for any application built after 2015. Use Playwright with headless Chromium. Do NOT use requests+BeautifulSoup as the primary crawl strategy – use it only as a fallback for non-JS pages.

```
from playwright.async_api import async_playwright, Page, BrowserContext

class JavaScriptCrawler:
    async def crawl(self, start_url: str, context: BrowserContext):
        page = await context.new_page()
        # Intercept ALL network requests – this finds hidden API endpoints
        page.on('request', self._on_request)
        page.on('response', self._on_response)
        await page.goto(start_url, wait_until='networkidle', timeout=30000)
        # Critical for React/Vue/Angular – wait for deferred renders
        await page.wait_for_load_state('networkidle')
        await asyncio.sleep(1)
        links = await self._extract_links(page)
        forms = await self._extract_forms(page)
        api_calls = self._extract_api_calls_from_log()
        await page.close()
        return CrawlResult(links=links, forms=forms, api_calls=api_calls)
```

◆ 1.3 SPA Route Discovery

Single-page apps change routes without making new HTTP requests. Inject a history API listener before any navigation to capture all route changes.

```
async def _discover_spa_routes(self, page: Page) -> list[str]:
    await page.evaluate('''
    window.__awap_routes = [];
    const origPush = history.pushState;
    history.pushState = function(...args) {
    window.__awap_routes.push(window.location.href);
    return origPush.apply(this, args);
    };
    window.addEventListener('hashchange', () =>
    window.__awap_routes.push(window.location.href));
    ''')
    nav_selectors = ['a[href]', 'button', '[data-route]', '[ng-click]', '@click']
    for sel in nav_selectors:
        for el in (await page.query_selector_all(sel))[:20]:
            try:
                await el.click(timeout=2000)
                await page.wait_for_load_state('networkidle', timeout=3000)
            except: continue
    routes = await page.evaluate('window.__awap_routes')
```

```
return [r for r in routes if self._is_in_scope(r)]
```

◆ 1.4 Preventing Infinite Crawl Loops

This is the most common crawler bug in AI-generated code. Without pattern-based deduplication, the crawler loops forever on paginated content.

```
class URLNormalizer:
    IGNOREABLE_PARAMS = {'utm_source', 'utm_medium', 'fbclid', '_ga', 'ref', 'ts', 'cb'}

    @classmethod
    def fingerprint(cls, url: str) -> str:
        # /user/1 and /user/2 get the SAME fingerprint - same attack surface
        parsed = urlparse(url)
        path = re.sub(r'/[0-9a-f-]{8,}/|$)', r'/{id}\1', parsed.path)
        path = re.sub(r'/\d+/$)', r'/{id}\1', path)
        return hashlib.md5(f'{parsed.netloc}{path}'.encode()).hexdigest()

class CrawlQueue:
    def should_visit(self, url: str, depth: int) -> bool:
        if depth > self.max_depth: return False
        if URLNormalizer.normalize(url) in self.visited_normalized: return False
        fp = URLNormalizer.fingerprint(url)
        # Max 3 variations of same path pattern - prevents /user/1..10000
        if self.visited_fingerprints.get(fp, 0) >= 3: return False
        return True
```

◆ 1.5 Domain Scope Enforcement

```
class ScopeEnforcer:
    def is_in_scope(self, url: str) -> bool:
        try:
            host = urlparse(url).hostname or ''
            # HARD BLOCK: internal IPs, cloud metadata, localhost
            if self._is_internal_ip(host): return False
            in_scope = any(p.match(host) for p in self.in_scope_patterns)
            out_scope = any(p.match(host) for p in self.out_of_scope_patterns)
            return in_scope and not out_scope
        except Exception:
            return False # Fail-closed on parse errors

    def _is_internal_ip(self, host: str) -> bool:
        import ipaddress
        try:
            ip = ipaddress.ip_address(host)
            return ip.is_private or ip.is_loopback or str(ip).startswith('169.254.')
        except ValueError:
            return host in ['localhost', 'metadata.google.internal', '169.254.169.254']
```

◆ 1.6 Crawl Priority Scoring

```
class CrawlPrioritizer:
    HIGH_VALUE = ['/api/', '/admin/', '/upload', '/graphql', '/exec', '/query']
    LOW_VALUE = ['/static/', '/assets/', '/images/', '/fonts/', '/favicon']

    @classmethod
    def score(cls, url: str, depth: int) -> int:
        base = depth * 10 # Deeper = lower priority
        path = urlparse(url).path.lower()
        for h in cls.HIGH_VALUE:
            if h in path: base -= 20 # Boost high-value paths
        for l in cls.LOW_VALUE:
            if l in path: base += 50 # Deprioritize static assets
```

```
return max(0, base)
```

■■■ 2. ENDPOINT AND PARAMETER DISCOVERY ENGINE ■■■

◆ 2.1 Multi-Source Endpoint Extraction

Endpoints must be harvested from five distinct sources. Missing any source means missing attack surface.

```
async def discover_all(self, crawl_result, js_files) -> list[Endpoint]:
    endpoints = []
    endpoints.extend(self._from_links(crawl_result.links)) # Source 1
    endpoints.extend(self._from_forms(crawl_result.forms)) # Source 2
    endpoints.extend(self._from_network_log(crawl_result.requests)) # Source 3
    for js_url in js_files:
        js = await self._fetch_js(js_url)
        endpoints.extend(self._from_javascript(js)) # Source 4
    endpoints.extend(await self._from_bruteforce(crawl_result.base_url)) # Source 5
    return self._deduplicate(endpoints)
```

◆ 2.2 JavaScript Endpoint Extraction

This is where most scanners miss large portions of the attack surface. Modern apps route everything through JS bundles. Use AST-aware regex patterns.

```
ENDPOINT_PATTERNS = [
    # fetch() and axios calls
    re.compile(r'(?:(?:fetch|axios\.get|axios\.post)\s*\(\s*["'](/[\\w\\-./{}?=%&]+)["']\s*,|
    # API path string literals
    re.compile(r'["'](?:/([\\w\\-./{}?=%&]+)|api|v\d|rest|graphql)[\\w\\-./{}?=%&]+["']\s*,|
    # Object property assignments
    re.compile(r'(?:(?:url|endpoint|path|apiUrl)\s*[:=]\s*["'](/[\\w\\-./{}?=%&]+)["']\s*,|
]

SECRET_PATTERNS = [
    re.compile(r'(?:(?:api|_|)?key|token|secret)\s*[:=]\s*["'](/[\\w\\-./+=}{20,})["']\s*, re.I),
    re.compile(r'AKIA[0-9A-Z]{16}'), # AWS Access Key
    re.compile(r'eyJ[a-zA-Z0-9_-]+\.[a-zA-Z0-9_-]+\.[a-zA-Z0-9_-]+'), # JWT
]
```

◆ 2.3 Parameter Storage Schema

```
class ParamLocation(Enum):
    URL_QUERY = 'url_query'
    URL_PATH = 'url_path'
    BODY_FORM = 'body_form'
    BODY_JSON = 'body_json'
    HEADER = 'header'
    COOKIE = 'cookie'
    GRAPHQL = 'graphql'

@dataclass
class Parameter:
    name: str
    location: ParamLocation
    original_value: str
    endpoint_id: str
    data_type: str = 'string'
    is_reflected: bool = False
    reflection_context: str = None
    waf_detected: bool = False
    encoding_required: list[str] = field(default_factory=list)
```

◆ 2.4 Baseline Profiling Before Attack

Before sending any attack payload, send 3 benign requests to establish a baseline. This is critical for anomaly detection and false positive reduction.

```
async def profile(self, endpoint, param, http_client) -> ParameterProfile:
    canary = f'AWAP{id(param):x}' # No special chars – pure probe
    response = await http_client.inject(endpoint, param, payload=canary)
    profile = ParameterProfile(
        baseline_status=response.status_code,
        baseline_length=len(response.body),
        baseline_time=response.elapsed_ms,
    )
    if canary in response.text:
        profile.is_reflected = True
    profile.reflection_context = self._detect_context(response.text, canary)
    # Probe for SQL context with single-quote
    sql_resp = await http_client.inject(endpoint, param, payload="")
    if self._has_sql_error(sql_resp.text):
        profile.likely_sql_context = True
    return profile
```

■■■ 3. ATTACK EXECUTION ENGINE ■■■

◆ 3.1 Self-Registering Module Pattern

```
_MODULE_REGISTRY: dict[str, type['AttackModule']] = {}

def register_module(cls):
    _MODULE_REGISTRY[cls.module_id] = cls
    return cls

class AttackModule(ABC):
    module_id: ClassVar[str]
    vuln_class: ClassVar[str]
    severity: ClassVar[str]
    safe_to_run_in_prod: ClassVar[bool] = True

    @abstractmethod
    async def run(self, endpoint, param, profile) -> list[Finding]: ...

    @abstractmethod
    async def verify(self, finding) -> bool: ...

    def load_all_modules():
        import awap.modules as pkg, pkgutil, importlib
        for _, name, _ in pkgutil.iter_modules(pkg.__path__):
            importlib.import_module(f'awap.modules.{name}')
```

◆ 3.2 Safe Payload Injection – All Parameter Locations

```
async def inject(self, endpoint, param, payload) -> HTTPResponse:
    if param.location == ParamLocation.URL_QUERY:
        url = inject_url_param(endpoint.url, param.name, payload)
        return await self._send(endpoint.method, url)

    elif param.location == ParamLocation.BODY_JSON:
        body = copy.deepcopy(endpoint.base_body_json or {})
        self._set_nested(body, param.name, payload) # Supports 'user.profile.name'
        return await self._send(endpoint.method, endpoint.url,
            headers={'Content-Type': 'application/json'}, body=json.dumps(body))

    elif param.location == ParamLocation.COOKIE:
        cookie = inject_cookie(endpoint.base_cookies, param.name, payload)
        return await self._send(endpoint.method, endpoint.url,
            headers={'Cookie': cookie})

    elif param.location == ParamLocation.URL_PATH:
        url = endpoint.url.replace(f'{{{param.name}}}', payload)
        return await self._send(endpoint.method, url)

    # ... handle BODY_FORM, HEADER, GraphQL similarly
```

◆ 3.3 Always Inject Valid Values for Non-Targeted Parameters

■ Most AI generators miss this. Sending ONLY the attack payload breaks multi-param endpoints. The server rejects the request before it hits the vulnerable field.

```
# WRONG – sends only the attack payload:
params = {param_to_attack.name: payload}

# CORRECT – send valid values for all other params too:
params = {p.name: p.original_value for p in endpoint.all_params}
params[param_to_attack.name] = payload # Overwrite only the target
```


4. PAYLOAD MANAGEMENT SYSTEM

4.1 Payload Library Structure

```
payloads/
  ■■■ sqli/
  ■ ■■■ error_based.txt # ' " ' ; "-- ' OR 1=1--
  ■ ■■■ blind_boolean.txt # ' AND 1=1-- ' AND 1=2--
  ■ ■■■ time_based.txt # ' ; WAITFOR DELAY '0:0:5'--
  ■ ■■■ oob.txt # DNS/HTTP callback payloads
  ■■■ xss/
  ■ ■■■ reflected.txt
  ■ ■■■ stored.txt
  ■ ■■■ dom.txt
  ■ ■■■ csp_bypass.txt
  ■■■ srf/
  ■ ■■■ internal_probes.txt # 169.254.169.254 192.168.x.x
  ■ ■■■ cloud_metadata.txt # AWS/GCP/Azure metadata URLs
  ■ ■■■ protocol_wrappers.txt # gopher:// dict:// file://
  ■■■ ssti/
  ■ ■■■ jinja2.txt # {{7*7}} {{config}}
  ■ ■■■ twig.txt # {{7*7}} {_self.env.registerUndefinedFilterCallback}}
  ■ ■■■ freemarker.txt
  ■■■ waf_bypass/
  ■■■ cloudflare.txt
  ■■■ generic.txt
```

4.2 Mutation Engine – WAF Bypass Generation

```
class PayloadMutator:
    @staticmethod
    def sql_comment_injection(p: str) -> str:
        # Bypass keyword-matching WAFs: SELECT → SE/**/LECT
        for kw in ['SELECT', 'UNION', 'FROM', 'WHERE', 'AND', 'OR']:
            p = re.sub(kw, f'{{kw[2:]}}/**/{{kw[2:]}}', p, flags=re.IGNORECASE)
        return p

    @staticmethod
    def shell_space_bypass(p: str) -> str:
        return p.replace(' ', '${IFS}') # Bypass space filters in OS command injection

    @classmethod
    def generate_variants(cls, payload: str, vuln_class: str) -> list[str]:
        variants = [payload]
        if vuln_class == 'XSS':
            variants += [url_encode(payload), double_url_encode(payload),
                          unicode_escape_js(payload), mixed_case(payload)]
        elif vuln_class == 'SQLI':
            variants += [url_encode(payload), cls.sql_comment_injection(payload),
                          payload.replace('"', "%27"), mixed_case(payload)]
        # Deduplicate preserving order
        seen = set()
        return [v for v in variants if not (v in seen or seen.add(v))]
```

■■■ 5. RESPONSE ANALYSIS ENGINE ■■■

◆ 5.1 Six-Layer Analysis

```
async def analyze(self, response, payload, param, vuln_class) -> AnalysisResult:
    signals = []
    signals.append(self._check_error_patterns(response, vuln_class)) # Layer 1
    signals.append(self._check_reflection(response, payload)) # Layer 2
    signals.append(self._check_status_anomaly(response)) # Layer 3
    signals.append(self._check_length_anomaly(response)) # Layer 4
    signals.append(self._check_timing_anomaly(response)) # Layer 5
    signals.append(self._check_structural_changes(response)) # Layer 6
    signals = [s for s in signals if s is not None]
    return AnalysisResult(
        signals=signals,
        confidence=self._noisy_or_confidence(signals),
        is_likely_vulnerable=self._noisy_or_confidence(signals) > 0.6,
    )

def _noisy_or_confidence(self, signals: list[Signal]) -> float:
    # P(at least one signal correct) = 1 - product(1 - P(each))
    p_none = 1.0
    for s in signals: p_none *= (1.0 - s.confidence)
    return 1.0 - p_none
```

◆ 5.2 Timing Anomaly Detection

```
def _check_timing_anomaly(self, response) -> Signal | None:
    if self.baseline.mean_response_time is None: return None
    # Threshold: baseline mean + 3 sigma, minimum 4 seconds absolute
    threshold = max(
        self.baseline.mean_response_time + 3 * self.baseline.std_response_time,
        4000,
    )
    if response.elapsed_ms > threshold:
        return Signal(type='timing_anomaly', confidence=0.75,
            evidence={'elapsed_ms': response.elapsed_ms,
                'threshold_ms': threshold})
    return None
```

◆ 5.3 False Positive Filter

```
def is_false_positive(self, result, response) -> bool:
    # Generic 404/500 error pages are not vulnerabilities
    body_lower = response.text.lower()
    if response.status_code in [400, 404, 500]:
        if any(fp in body_lower for fp in ['page not found', '404 not found']): return True
    # Length-change alone is NOT a finding
    if len(result.signals)==1 and result.signals[0].type=='length_anomaly': return True
    # Timing: require 3 consistent anomalies, not just 1
    if all(s.type=='timing_anomaly' for s in result.signals) and len(result.signals)<3:
        return True
    return False
```

■■■ 6. ASYNCHRONOUS SCANNING SYSTEM ■■■

■ NEVER use `asyncio.gather(*all_tasks)` for scan tasks. This launches all tasks simultaneously, exhausts connections, and crashes the target. Always use a semaphore-controlled worker pool.

◆ 6.1 Semaphore Worker Pool

```
class ScanWorkerPool:
    def __init__(self, config):
        # Global semaphore: total concurrent requests across ALL targets
        self._global_sem = Semaphore(config.max_concurrent_requests)
        # Per-target semaphores: max connections per single target
        self._target_sems = {}
        self._rate_limiter = RateLimiter(config.requests_per_second)
        self._queue = asyncio.Queue(maxsize=config.queue_size)

    async def _execute_with_limits(self, task):
        host = urlparse(task.endpoint.url).netloc
        if host not in self._target_sems:
            self._target_sems[host] = Semaphore(self.config.max_per_target)
        async with self._target_sems[host]:
            async with self._global_sem:
                await self._rate_limiter.acquire() # Token bucket
            return await task.execute()
```

◆ 6.2 Token Bucket Rate Limiter

```
class RateLimiter:
    def __init__(self, rate: float): # rate = requests/second
        self.rate = rate
        self.tokens = rate
        self.last = time.monotonic()
        self._lock = asyncio.Lock()

    async def acquire(self):
        async with self._lock:
            now = time.monotonic()
            self.tokens = min(self.rate, self.tokens + (now-self.last)*self.rate)
            self.last = now
            if self.tokens >= 1:
                self.tokens -= 1
            else:
                wait = (1 - self.tokens) / self.rate
                await asyncio.sleep(wait)
            self.tokens = 0
```

7. SCAN CONTROL AND SAFETY SYSTEM

7.1 Scan State Machine

```
# Valid state transitions:
# CREATED → SCOPE_VERIFIED → RECON → CRAWL → MAPPING → ATTACK → ANALYSIS → COMPLETE
# ↑
# PAUSED ← → ABORTED

class ScanController:
    async def checkpoint(self):
        '''Call at start of every task. Blocks on pause, raises on abort.'''
        if self._abort_event.is_set():
            raise ScanAbortedException()
        await self._pause_event.wait() # Block until unpaused
        if self._abort_event.is_set():
            raise ScanAbortedException()

    async def abort(self):
        self._abort_event.set()
        self._pause_event.set() # Unblock waiting workers so they can check abort
```

7.2 Anti-DoS Guard – Hard Limits

```
class AntiDoSGuard:
    HARD_MAX_RPS = 50 # Never exceed regardless of user config
    HARD_MAX_CONNECTIONS = 50
    ERROR_RATE_THRESHOLD = 0.30 # >30% errors → auto-throttle

    async def on_response(self, response, rate_limiter):
        if response.status_code == 429:
            self._consecutive_429s += 1
            backoff = min(2 ** self._consecutive_429s, 60) # Max 60s
            await asyncio.sleep(backoff)
            rate_limiter.rate = max(1, rate_limiter.rate * 0.5) # Halve rate
            if len(self._error_window) >= 20:
                error_rate = sum(self._error_window) / len(self._error_window)
                if error_rate > self.ERROR_RATE_THRESHOLD:
                    rate_limiter.rate = max(1, rate_limiter.rate * 0.7)
```

8. SCAN DATA STORAGE DESIGN

8.1 PostgreSQL Schema

```
-- Separate high-query findings table from high-volume request log
CREATE TABLE findings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  scan_id UUID NOT NULL REFERENCES scans(id) ON DELETE CASCADE,
  endpoint_id UUID REFERENCES endpoints(id),
  vuln_class VARCHAR(30) NOT NULL,
  severity VARCHAR(10) NOT NULL,
  cvss_score NUMERIC(3,1),
  payload TEXT,
  confidence NUMERIC(3,2),
  verified BOOLEAN DEFAULT FALSE,
  false_positive BOOLEAN DEFAULT FALSE,
  evidence JSONB,
  request_raw TEXT,
  response_raw TEXT
);
CREATE INDEX idx_findings_scan_id ON findings(scan_id);
CREATE INDEX idx_findings_severity ON findings(severity) WHERE false_positive=FALSE;

-- Partition request_log by scan_id - it grows to millions of rows
CREATE TABLE request_log (
  id BIGSERIAL,
  scan_id UUID NOT NULL,
  method VARCHAR(10),
  url TEXT,
  response_status INT,
  response_body TEXT, -- Truncate at 1MB before INSERT
  elapsed_ms INT,
  logged_at TIMESTAMPTZ DEFAULT NOW()
) PARTITION BY HASH(scan_id);
```

8.2 Storage Best Practices

- Never store full response bodies in the findings table – use request_log with FK references
- Truncate response bodies to 1MB before INSERT
- Deduplicate findings at INSERT time using (scan_id, vuln_class, endpoint_id, parameter_id) unique check
- Use JSONB for evidence blobs – allows GIN indexing for fast evidence search

■■■ 9. VULNERABILITY RESULT VALIDATION ■■■

◆ 9.1 Multi-Trial Verification

```
REQUIRED_CONFIRMATIONS = {
'SQLI_ERROR': 1, # DB error signature = definitive
'SQLI_BLIND': 3, # Boolean-blind needs 3 consistent trials
'SQLI_TIME': 3, # Time-based needs 3 consistent timing anomalies
'XSS_REFLECTED': 2,
'SSRF': 1, # OOB callback = definitive
'CMDI': 2,
}

async def verify(self, finding, module) -> VerificationResult:
    required = REQUIRED_CONFIRMATIONS.get(f'{finding.vuln_class}_{finding.sub_type}', 2)
    confirmations = 0
    for trial in range(required + 1):
        await asyncio.sleep(1.0) # Gap between trials
        if await module.verify(finding):
            confirmations += 1
    return VerificationResult(
        verified=confirmations >= required,
        confidence_adjustment=confirmations / (required + 1),
    )
```

◆ 9.2 OOB Callback Server

```
class OOBServer:
    def generate_token(self, scan_id, finding_id) -> str:
        token = sha256(f'{scan_id}{finding_id}{time.time()}'.encode()).hexdigest()[:12]
        self._callbacks[token] = asyncio.Event()
        return token

    def get_http_url(self, token): return f'http://{token}.{self.domain}'
    def get_dns_host(self, token): return f'{token}.{self.domain}'

    async def wait_for_callback(self, token, timeout=30.0) -> bool:
        try:
            await asyncio.wait_for(self._callbacks[token].wait(), timeout)
            return True
        except asyncio.TimeoutError:
            return False

    async def on_dns_query(self, query):
        token = self._extract_token(query.name)
        if token in self._callbacks:
            self._received[token] = OOBCallback(type='dns', source_ip=query.source_ip)
            self._callbacks[token].set() # Unblock wait_for_callback
```

■■■ 10. ERROR HANDLING AND STABILITY ■■■

◆ 10.1 Robust HTTP Client with Retry

```
import httpx
from tenacity import retry, stop_after_attempt, wait_exponential

class RobustHTTPClient:
    def __init__(self):
        self.client = httpx.AsyncClient(
            verify=False, # Scanning self-signed certs is expected
            follow_redirects=True,
            timeout=httpx.Timeout(connect=10, read=30, write=10, pool=60),
            limits=httpx.Limits(max_keepalive_connections=20, max_connections=50),
        )

    @retry(stop=stop_after_attempt(3),
          wait=wait_exponential(multiplier=1, min=1, max=10),
          retry=retry_if_exception_type((ConnectTimeout, ReadTimeout, ConnectError)))
    async def request(self, method, url, **kwargs):
        try:
            return await self.client.request(method, url, **kwargs)
        except httpx.TooManyRedirects:
            return HTTPResponse.error(url, 0, 'too_many_redirects') # Don't retry
        except httpx.InvalidURL:
            return HTTPResponse.error(url, 0, 'invalid_url') # Don't retry
```

◆ 10.2 Scan Checkpoint / Resume

```
class ScanRecoveryManager:
    async def save_checkpoint(self, scan_id, state):
        await self.db.execute('''
            INSERT INTO scan_checkpoints (scan_id, checkpoint_data)
            VALUES ($1, $2)
            ON CONFLICT (scan_id) DO UPDATE SET checkpoint_data=$2
            ''', scan_id, json.dumps(state.to_dict()))

    async def resume_scan(self, scan_id) -> bool:
        chk = await self.load_checkpoint(scan_id)
        if not chk: return False
        # Filter completed_endpoint_ids out of the task queue
        # before re-launching the scan engine
        return True
```

■■■ 11. PERFORMANCE OPTIMIZATION ■■■

◆ 11.1 Response Cache

```
class ResponseCache:
    def get(self, method, url, headers) -> CachedResponse | None:
        if method != 'GET': return None # NEVER cache non-GET requests
        key = sha256(f'{method}:{url}:{sorted(stable_headers.items())}'.encode()).hexdigest()
        cached = self._store.get(key)
        if cached and not cached.is_expired(): return cached
        return None

    def set(self, method, url, headers, response, ttl=300):
        if method != 'GET': return
        if len(response.body) > 5 * 1024 * 1024: return # Skip large responses
        # LRU eviction when cache is full
        while self._current_size + len(response.body) > self._max_bytes:
            oldest = next(iter(self._store))
            self._current_size -= len(self._store.pop(oldest).body)
```

◆ 11.2 Endpoint Risk Scoring – Prioritize Attack Order

```
def score_endpoint(self, endpoint, params) -> float:
    score = 0.0
    # File extension risk
    ext = endpoint.url.split('.')[1].lower()
    score += {'php':0.3,'asp':0.3,'aspx':0.3,'jsp':0.3}.get(ext, 0.0)
    # Parameter count
    score += min(len(params) * 0.1, 0.5)
    # Reflected params = XSS candidates
    score += sum(0.3 for p in params if p.is_reflected)
    # Integer params = SQLi candidates
    score += sum(0.2 for p in params if p.data_type == 'integer')
    # High-risk path keywords
    risky = ['admin','upload','exec','query','search','proxy','redirect','eval']
    if any(kw in endpoint.url.lower() for kw in risky): score += 0.2
    return min(score, 1.0)
```


■■■ 12. DEVELOPER IMPLEMENTATION NOTES ■■■

◆ 12.1 Top 10 AI Code Generator Mistakes

Mistake 1: Using requests library for crawling SPAs

Fix: Use Playwright with wait_until='networkidle'. requests misses ALL JS-rendered content.

Mistake 2: asyncio.gather(*all_tasks) without limits

Fix: Use a Semaphore-controlled worker pool. gather() on 1000 tasks crashes everything.

Mistake 3: URL dedup by string equality alone

Fix: /user/1 and /user/2 are the same endpoint. Use path pattern fingerprinting.

Mistake 4: Only injecting the attack payload, ignoring other params

Fix: Always carry original_value for all params. Inject them alongside the attack payload.

Mistake 5: No response baseline before attacking

Fix: Send 3 benign requests first. Without baseline, every 500 error looks like a vuln.

Mistake 6: Scope check AFTER making the request

Fix: Scope enforcement must happen BEFORE any network connection opens. Fail-closed.

Mistake 7: Hardcoding User-Agent as 'python-requests'

Fix: Immediately blocked by most WAFs. Rotate through realistic browser UA strings.

Mistake 8: No CSRF token extraction for POST forms

Fix: Extract CSRF tokens from the page before submitting forms.

Mistake 9: Storing full response bodies in findings table

Fix: Use request_log table (partitioned). Findings table must stay small and fast.

Mistake 10: Timing verification with only 1 trial

Fix: Time-based blind SQLi requires 3 consistent timing anomalies to confirm.

◆ 12.2 Required User-Agent Pool

```
USER_AGENTS = [  
    'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 Chrome/124.0.0.0',  
    'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 Chrome/124.0.0.0',  
    'Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:124.0) Gecko/20100101 Firefox/124.0',  
    'Mozilla/5.0 (Macintosh; Intel Mac OS X 14_4_1) AppleWebKit/605.1.15 Safari/17.4.1',  
]  
import random  
def get_ua(): return random.choice(USER_AGENTS)
```

◆ 12.3 CSRF Token Extraction Pattern

```
async def extract_csrf_token(page: Page) -> dict[str, str]:  
    # Method 1: Hidden form inputs  
    tokens = await page.evaluate('''  
    () => {  
    const r = {};  
    document.querySelectorAll('input[type=hidden]').forEach(el => {  
    if (/csrf|token|nonce|wpnonce/i.test(el.name)) r[el.name] = el.value;  
    });  
    return r;  
    }''
```

```
''')
# Method 2: Meta tags (Laravel, Rails, Django)
meta = await page.evaluate('''
() => {
const m = document.querySelector('meta[name="csrf-token"],meta[name="_token"]');
return m ? {[m.getAttribute('name')]: m.getAttribute('content')} : {};
}
''')
tokens.update(meta)
return tokens
```

■■■ IMPLEMENTATION CHECKLIST ■■■

Verify each item before considering the engine production-ready:

■ Crawler

- [] Playwright headless Chromium with wait_until='networkidle'
- [] Session persistence via storage_state across all pages
- [] URL normalization + path pattern fingerprinting deduplication
- [] Scope enforcement BEFORE requests, fail-closed on errors
- [] SPA route discovery via history API injection
- [] All XHR/Fetch intercepted and logged
- [] CSRF token extraction for form submissions

■ Attack Engine

- [] Module self-registration via @register_module decorator
- [] 3-request baseline established before any attack payloads
- [] Valid original values injected for all non-targeted parameters
- [] All 6 parameter locations handled (URL, form, JSON, header, cookie, path)
- [] Token bucket rate limiter with hard caps (50 rps, 50 connections)
- [] Checkpoint/resume on crash via scan_checkpoints table

■ Response Analysis

- [] Six-layer analysis (error, reflection, status, length, timing, structural)
- [] Noisy-OR confidence aggregation across signals
- [] False positive filter (generic error pages, low-signal-only detections)
- [] 3-trial confirmation required for timing-based findings

■ Storage

- [] Findings deduplicated at INSERT level
- [] request_log partitioned by scan_id
- [] Response bodies truncated at 1MB before INSERT
- [] All tables indexed on scan_id

■ Safety

- [] RFC1918 + cloud metadata IP hard-block
- [] 429 exponential backoff (max 60s)
- [] Error-rate-triggered auto-throttle (>30% errors → reduce rate)
- [] Scan pause/abort with immediate effect on all workers
- [] Full audit log for scan start/stop/auth events

FOR AUTHORIZED SECURITY RESEARCH USE ONLY · This document covers engine implementation only. ·
Architecture, UI, and deployment are in companion documents.